SpendWise – Personal Finance Tracker

INTRODUCTION:

SpendWise is a versatile and user-friendly full-stack web application designed to streamline personal financial management. Suitable for tracking daily expenses, managing income, setting category-wise budgets, and analyzing spending behavior, it offers a comprehensive suite of tools — including interactive dashboards, pie charts, line charts, and CSV data export — that can be navigated through an intuitive and premium dark/light themed interface. Users can fully manage multiple financial accounts (Bank, Cash, Digital Wallet) while robust security features like JWT-based authentication and Bcrypt password hashing protect data integrity. SpendWise also features real-time budget threshold alerts that automatically notify users when their spending approaches predefined limits. With built-in data visualization tools powered by Recharts, users can generate visual spending reports and export transaction history for deeper offline analysis. Its flexibility, modern tech stack, and powerful features make it an ideal choice for students, professionals, and anyone seeking an efficient personal finance tracking solution.

SCENARIO Based Case Study:

Meet Priya, a college student juggling academics and part-time work who values financial discipline and organization in her daily life. Priya is responsible for managing her limited monthly budget, tracking expenses across multiple categories like food, transport, and entertainment, and ensuring she saves enough for upcoming semester fees.

Priya discovers SpendWise, a comprehensive web application designed to streamline personal finance management. Intrigued by its features, Priya decides to explore how SpendWise can help her take control of her spending habits.

User Registration and Authentication: Priya registers an account on SpendWise, providing her name, email, and a secure password. She logs in securely using JWT-based authentication, ensuring that her financial data remains private and protected.

Account Management: SpendWise allows Priya to create multiple financial accounts — she sets up a “Bank Account” for her savings, a “Cash” account for daily spending, and a “Digital Wallet” for online transactions. She can track balances independently for each account.

Transaction Logging: Priya uses SpendWise to record every income and expense transaction. When she receives her part-time salary, she logs it as income under the “Salary” category. When

she buys lunch, she logs it as an expense under “Food & Dining.” Each transaction is timestamped and categorized automatically.

Budget Management: SpendWise allows Priya to set monthly budget limits for specific categories. She sets ₹3,000 for Food & Dining and ₹1,500 for Entertainment. The system tracks her spending against these limits with visual progress bars.

Smart Alerts: When Priya’s food spending reaches 90% of her ₹3,000 budget, SpendWise triggers an automatic alert on her dashboard, warning her to slow down before exceeding her limit.

Dashboard Analytics: SpendWise provides Priya with a comprehensive dashboard featuring summary cards (Total Income, Total Expenses, Net Balance), a category-wise pie chart showing expense distribution, and a line chart tracking daily spending trends over time.

CSV Export: At the end of the month, Priya exports her complete transaction history as a CSV file for her personal records and shares it with her parents for transparency.

Priya’s Experience: Thanks to SpendWise, Priya can now manage her personal finances more efficiently and effectively. She can focus on her studies while maintaining complete visibility over her spending patterns, budget adherence, and savings growth. SpendWise has become an indispensable tool in Priya’s daily life, helping her achieve her financial goals with ease and confidence.

SYSTEM REQUIREMENTS:

Software Requirements:

To ensure smooth development, deployment, and usage of the SpendWise Web-Application, certain system prerequisites must be met. These requirements are categorized into software, setup, and hardware specifications, all of which contribute to building a robust and scalable personal finance platform.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- **Operating System:** Windows 10/11, macOS, or Linux — Supports cross-platform development and testing.
- **Node.js (v18 or above):** Provides the JavaScript runtime environment for both the frontend (Vite) and the backend (Express).

- **npm (v9 or above):** A package manager required to install dependencies for both the React frontend and Node.js backend.
- **React.js (v19):** A JavaScript library for building dynamic and responsive user interfaces.
- **Express.js:** A minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications.
- **Mongoose:** A MongoDB object modeling tool (ODM) designed to work in an asynchronous environment.
- **Tailwind CSS (v4):** A utility-first CSS framework for rapid UI development with a premium aesthetic.
- **Browser:** Google Chrome / Firefox / Edge (latest version) — For rendering and testing the UI in real-time.
- **MongoDB Atlas:** A cloud-hosted NoSQL database used to store structured data related to users, transactions, budgets, accounts, and categories.
- **Postman:** Tool for testing APIs during development.
- **Visual Studio Code:** Preferred code editor with built-in Git and terminal support.

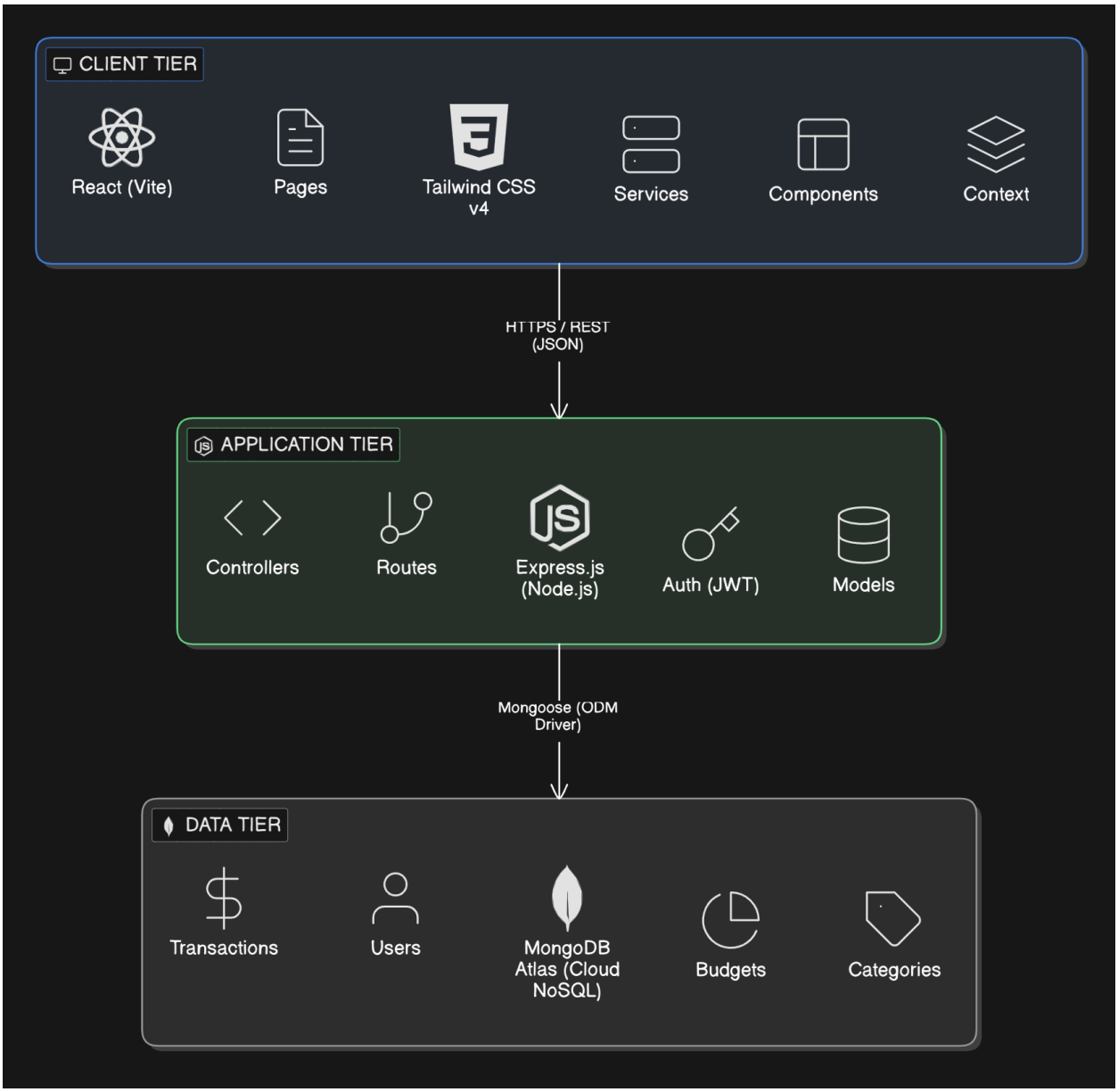
2. Hardware Requirements

Describes the minimum and recommended specifications needed to support the development and usage of the application.

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — Ensures fast compilation and multitasking during development.
- **RAM:** Minimum 8 GB (16 GB recommended) — For handling development servers, IDEs, and browser testing simultaneously.
- **Storage:** At least 1 GB free space — Required for package installations, MongoDB setup, and local project files.
- **Display:** 1366x768 or higher — Recommended for optimal coding experience and application layout visualization.

PROJECT ARCHITECTURE:

SpendWise follows a modern **three-tier architecture** separating the application into distinct layers:



TECHNICAL ARCHITECTURE:

In this architecture, the flow begins with a **User** interacting with the SpendWise frontend (a React application hosted on Vercel). The user performs actions such as logging in, adding transactions, or viewing their dashboard.

The SpendWise frontend then sends **HTTP requests** (via Axios) to the SpendWise backend (an Express.js application hosted on Render). Each request includes a **JWT token** in the Authorization header for security. The backend processes the request and validates the token.

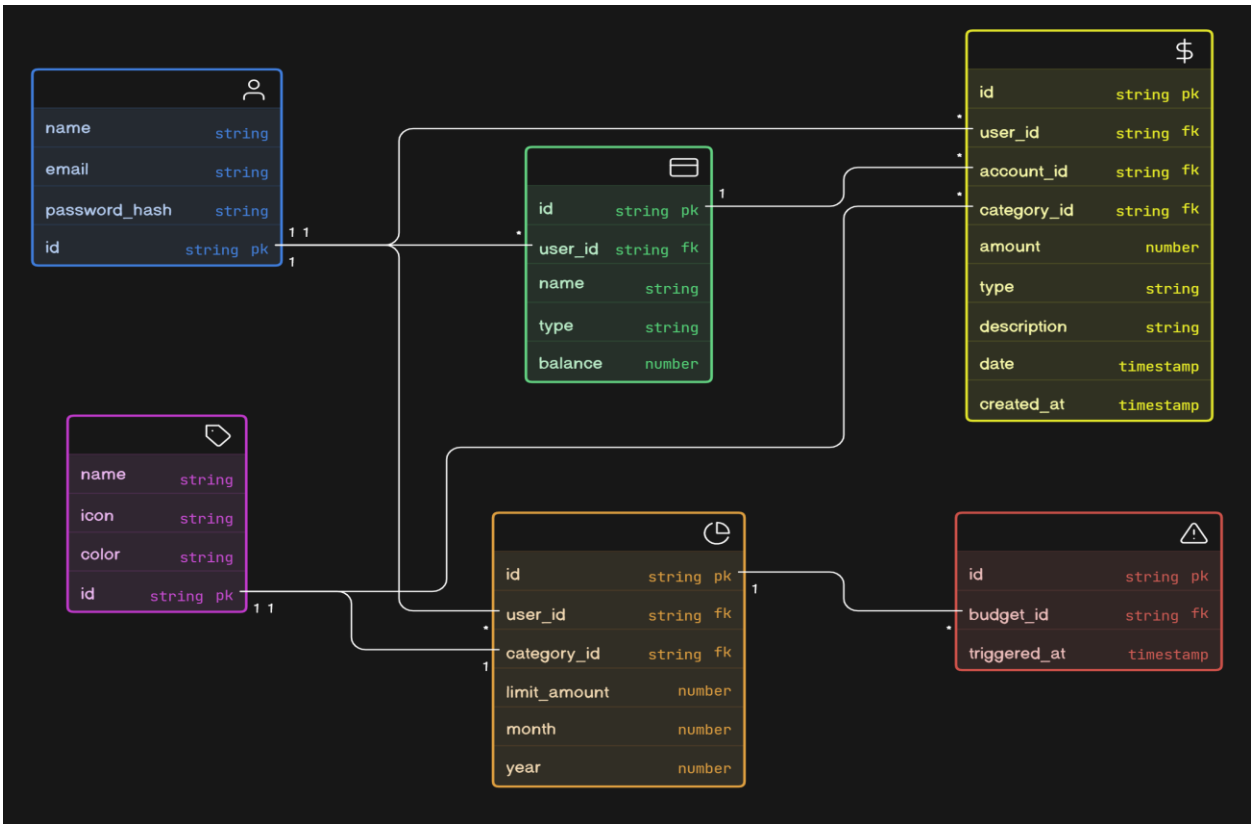
The backend communicates with **MongoDB Atlas** (a cloud-hosted NoSQL database) using the **Mongoose ODM** to read/write data. The database stores all user records, financial transactions, budget configurations, account details, and category definitions.

The backend applies **business logic** — such as calculating budget thresholds, aggregating dashboard statistics, and triggering budget alerts — through specialized **Controllers** before returning the processed response to the frontend.

Once the backend has processed the request, it sends a **JSON response** back to the React frontend. The frontend then updates its state and renders the appropriate UI components (charts, tables, alerts) to the user.

ER DIAGRAM:

The Entity-Relationship (ER) diagram for SpendWise represents the various entities and their relationships within the personal finance management system. Here is a description of the key components:



1. Entities:

- **User:** Represents registered users of the system, with attributes such as id, name, email, and password_hash.
- **Transaction:** Represents financial log entries (income/expense), with attributes such as id, amount, type, description, date, and created_at.
- **Budget:** Represents monthly spending limits set by users for specific categories, with attributes such as id, limit_amount, month, and year.
- **Account:** Represents financial accounts owned by users (Bank, Cash, Digital Wallet), with attributes such as id, name, type, and balance.
- **Category:** Represents predefined expense/income categories, with attributes such as id, name, icon, and color.
- **Alert:** Represents budget threshold triggers, with attributes such as budget_id and triggered_at.

2. Relationships:

- **User → Transaction:** One-to-Many. A user can log multiple transactions.
- **User → Budget:** One-to-Many. A user can set multiple budgets across categories.
- **User → Account:** One-to-Many. A user can own multiple financial accounts.
- **Transaction → Category:** Many-to-One. Each transaction is classified under one category.
- **Transaction → Account:** Many-to-One. Each transaction belongs to one account.
- **Budget → Category:** One-to-One. Each budget tracks spending for one category.

3. Attributes:

Each entity has its own set of attributes that describe its properties. For example, the Transaction entity has attributes such as id, user_id, account_id, category_id, amount, type, description, date, and created_at.

4. Primary Keys:

Each entity has a primary key that uniquely identifies each record. In MERN, the primary key is _id (MongoDB ObjectId), which is automatically handled by Mongoose.

5. Foreign Keys:

Foreign keys establish relationships between entities. For example, the Transaction entity has foreign keys user_id, account_id, and category_id linking it to the respective parent entities using Mongoose Schema.Types.ObjectId and ref.

Key Features:

The key features of SpendWise personal finance tracker include:

User Registration and Authentication: Users can create accounts using email and password. JWT-based authentication ensures secure access to all protected resources.

Multi-Account Management: Users can create and manage multiple financial accounts (Bank, Cash, Digital Wallet) with independent balance tracking.

Transaction Management: Users can add, edit, and delete income/expense transactions with category classification, timestamps, and optional descriptions.

Budget Management: Users can set monthly spending limits for specific categories. Progress bars visually indicate how much of each budget has been consumed.

Smart Alerts: The system automatically triggers alerts when spending reaches 90% of a defined budget limit, helping users avoid overspending.

Dashboard Analytics: A comprehensive dashboard displays Total Income, Total Expenses, and Net Balance through summary cards, a category-wise Pie Chart, and a daily spending Line Chart.

CSV Export: Users can export their complete transaction history as a downloadable CSV file for offline analysis and record-keeping.

Dark/Light Theme: A premium theme toggle allows users to switch between dark and light modes for comfortable viewing.

ROLES AND RESPONSIBILITIES:

User:

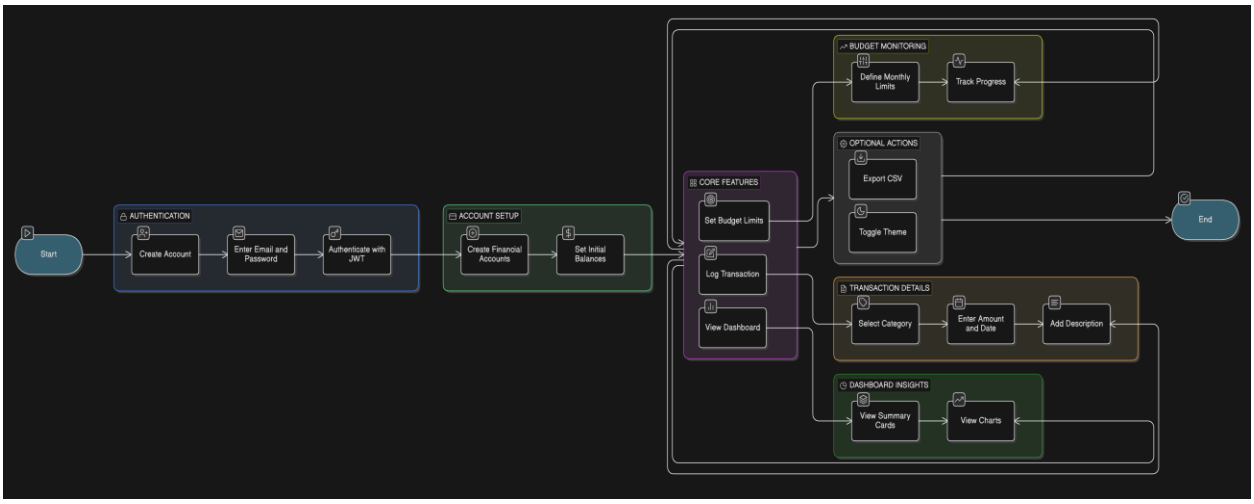
- **Registration and Account Management:** Users are responsible for creating an account, providing accurate personal information (name, email, password), and managing their profile securely.
- **Account Setup:** Users create and configure their financial accounts (Bank, Cash, Digital Wallet), setting initial balances and organizing their financial portfolio.
- **Transaction Logging:** Users are responsible for recording their income and expense transactions accurately, selecting appropriate categories, accounts, and dates.
- **Budget Configuration:** Users can set monthly spending limits for specific categories and monitor their progress through visual progress bars on the Budgets page.
- **Data Export:** Users can export their transaction history as CSV files for offline auditing, tax preparation, or sharing with financial advisors.

System (Automated):

- **Authentication & Security:** The system is responsible for hashing passwords using Bcryptjs, generating and validating JWT tokens, and protecting all API endpoints from unauthorized access.
- **Budget Alert Monitoring:** The system automatically calculates spending against defined budgets using aggregation pipelines and triggers warning alerts when thresholds are reached.
- **Dashboard Aggregation:** The system aggregates transaction data in real-time to compute total income, total expenses, net balance, category-wise breakdowns, and daily spending trends.
- **Data Integrity:** The system ensures that account balances are automatically updated via backend middleware/logic when transactions are added, edited, or deleted.

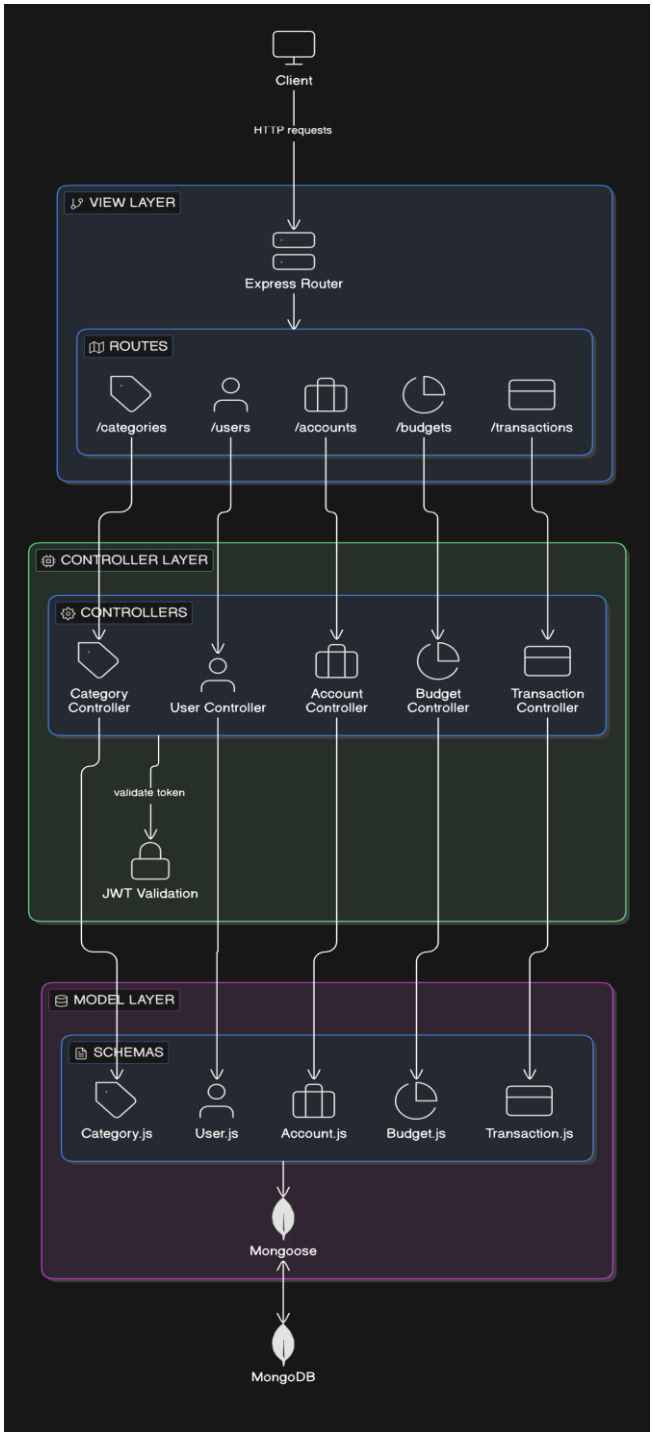
User Flow:

- **Registration & Login** – Create account with email/password, authenticate via JWT.
- **Account Setup** – Create financial accounts (Bank, Cash, Wallet) with initial balances.
- **Transaction Logging** – Record income/expense with category, amount, date, and description.
- **Budget Setting** – Define monthly limits per category, monitor via progress bars.
- **Dashboard Viewing** – View summary cards, pie charts, line charts for financial insights.
- **CSV Export** – Download complete transaction history for offline records.
- **Theme Toggle** – Switch between dark and light modes for comfortable viewing.



MVC PATTERN:

The SpendWise application follows the **Model-View-Controller (MVC)** architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.



Model Layer (Data Layer):

The Model layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database. The models are implemented using **Mongoose Schemas**. Key models include:

- User.js – Schema for user accounts
- Transaction.js – Schema for income/expense records
- Budget.js – Schema for monthly spending limits
- Account.js – Schema for financial accounts
- Category.js – Schema for transaction categories

Controller Layer:

The Controller layer acts as an intermediary between the view (routes) and the model. In our MERN stack, these are implemented as **Controller files** (e.g., transactionController.js). They receive incoming requests from the routes, validate the JWT token, process the input, perform database operations via Mongoose, and return a JSON response.

View Layer (Routing Layer):

In the context of the backend REST API, the View is implemented as the **routing layer** using Express.js Router. Various endpoints are defined in routes/ that determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

In the context of the frontend, the View is implemented as **React Pages and Components** that render the UI and handle user interactions.

Advantages of Using MVC in This Project:

- **Separation of Concerns:** Each layer has a clearly defined responsibility, improving readability and maintainability.
- **Scalability:** New features can be added easily by creating new routes, controllers, and models.
- **Reusability:** Business logic in controllers can be reused across multiple parts of the application.
- **Testing:** Each layer can be tested independently — backend via Vitest/Supertest, frontend via Vitest.

Project Setup And Configuration:

Creating project folder

1. Create a new folder with your project name (SpendWise).
2. Inside that folder create two new folders.
3. Name one as **frontend**.
4. Name another one as **backend**.
5. Now open that folder in VS Code.

Frontend setup (installing React app with Vite)

- Open the frontend folder in the terminal of VSCode.
    - npm create vite@latest . -- --template react
- Select React framework from the given options.
- Select JavaScript variant from the given options.
- Install all the packages:
    - npm install
- Install additional dependencies:
    - npm install react-router-dom axios recharts lucide-react framer-motion
    - npm install -D tailwindcss @tailwindcss/vite
- To start the React development server:
    - npm run dev

Backend setup (Node.js Environment)

- Open the backend folder in the terminal of VSCode.
    - npm init -y
- Install dependencies:
    - npm install express mongoose jsonwebtoken bcryptjs cors dotenv
    - npm install -D nodemon

- Create files and folders:
    - server.js – Application entry point
    - config/db.js – MongoDB connection setup using Mongoose
    - models/ – Mongoose schemas (User, Transaction, Budget, Account, Category)
    - controllers/ – Business logic for API endpoints
    - routes/ – API route handlers (Auth, Transaction, Budget, etc.)
    - middleware/ – JWT authentication middleware
- To start the backend server:
    - npm run dev (using nodemon)

BACKEND DEVELOPMENT:

Backend Structure:

```
backend/
├── config/
│   └── db.js            → MongoDB connection setup using Mongoose
├── controllers/
│   ├── accountController.js  → Logic for account CRUD
│   ├── authController.js     → Logic for registration and login
│   ├── budgetController.js   → Logic for budget tracking
│   ├── categoryController.js → Logic for category management
│   ├── dashboardController.js → Logic for dashboard aggregation
│   └── transactionController.js → Logic for transactions and CSV export
├── middleware/
│   └── authMiddleware.js     → JWT token verification logic
├── models/
│   ├── Account.js       → Mongoose schema for accounts
│   ├── Alert.js         → Mongoose schema for alerts
│   ├── Budget.js        → Mongoose schema for budgets
│   ├── Category.js      → Mongoose schema for categories
│   ├── Transaction.js   → Mongoose schema for transactions
│   └── User.js          → Mongoose schema for users
├── routes/
│   ├── accountRoutes.js      → Routes for account endpoints
│   ├── authRoutes.js         → Routes for auth endpoints
│   ├── budgetRoutes.js       → Routes for budget endpoints
│   ├── categoryRoutes.js     → Routes for category endpoints
│   ├── dashboardRoutes.js    → Routes for dashboard endpoints
│   └── transactionRoutes.js  → Routes for transaction endpoints
├── server.js            → Application entry point and server config
├── package.json         → Node.js dependencies
└── Procfile             → Production process manager config (Render)
```

Controllers & Routes:

- **authRoutes.js / authController.js** → Handles all authentication-related operations: user registration with Bcryptjs hashing, login with JWT token generation, and current user profile retrieval.
- **transactionRoutes.js / transactionController.js** → Handles all transaction-related operations: create, read, update, delete transactions; CSV export with category/account name resolution; automatic account balance updates.
- **budgetRoutes.js / budgetController.js** → Handles all budget-related operations: create/update monthly category budgets, track spending against limits, trigger alerts when thresholds are exceeded.
- **accountRoutes.js / accountController.js** → Handles all account-related operations: create, read, update, delete financial accounts with automatic balance tracking.
- **dashboardRoutes.js / dashboardController.js** → Handles dashboard data aggregation: compute total income/expenses/balance, generate category-wise pie chart data, and daily spending trend data for line charts.

Auth Middleware:

- **authMiddleware.js** → Middleware that extracts and validates the JWT token from the Authorization header of each request. It attaches the decoded user object to the request for protected endpoints.

Models:

- **User** (User.js) → Schema for user profile (name, email, password_hash).
- **Transaction** (Transaction.js) → Schema for financial records (amount, type, category_id, account_id).
- **Budget** (Budget.js) → Schema for category limits (limit_amount, month, year).
- **Account** (Account.js) → Schema for financial accounts (name, type, balance).
- **Category** (Category.js) → Schema for transaction groups (name, icon, color).

DATABASE DEVELOPMENT:

1. Configure MongoDB:

- **Install Mongoose**:
- npm install mongoose
- **Create database connection:**

The database connection is established using Mongoose's connect method. The connection string is managed via environment variables.

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGODB_URL);
    console.log(`✅ Connected to MongoDB: ${conn.connection.name}`);
  } catch (error) {
    console.error(`❌ Error: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

2. Create Schemas:

The MongoDB collections store documents matching the following Mongoose schemas:

1) Users Collection

{

name: { type: String, required: true },

email: { type: String, required: true, unique: true },

password_hash: { type: String, required: true },

created_at: { type: Date, default: Date.now }

}

2) Transactions Collection

{

user_id: { type: Schema.Types.ObjectId, ref: 'User' },

```
  account_id: { type: Schema.Types.ObjectId, ref: 'Account' },
  category_id: { type: Schema.Types.ObjectId, ref: 'Category' },
  amount: Number,
  type: String, // "income" | "expense"
  description: String,
  date: Date
}
```

3) Budgets Collection

```
{
  user_id: { type: Schema.Types.ObjectId, ref: 'User' },
  category_id: { type: Schema.Types.ObjectId, ref: 'Category' },
  limit_amount: Number,
  month: Number,
  year: Number
}
```

4) Accounts Collection

```
{
  user_id: { type: Schema.Types.ObjectId, ref: 'User' },
  name: String,
  type: { type: String, enum: ['bank', 'cash', 'wallet'] },
  balance: Number,
  created_at: { type: Date, default: Date.now }
}
```

5) Categories Collection

```
{
  user_id: { type: Schema.Types.ObjectId, ref: 'User' },
  name: String,
```

```
  type: { type: String, enum: ['income', 'expense'] },
  icon: String,
  color: String
}
```

6) Alerts Collection

```
{
  user_id: { type: Schema.Types.ObjectId, ref: 'User' },
  budget_id: { type: Schema.Types.ObjectId, ref: 'Budget' },
  threshold_pct: { type: Number, default: 90 },
  triggered_at: { type: Date, default: null }
}
```

4) Accounts Collection

```
{
  user_id: { type: Schema.Types.ObjectId, ref: 'User' },
  name: String,
  type: { type: String, enum: ['bank', 'cash', 'wallet'] },
  balance: Number,
  created_at: { type: Date, default: Date.now }
}
```

5) Categories Collection

```
{
  user_id: { type: Schema.Types.ObjectId, ref: 'User' },
  name: String,
  type: { type: String, enum: ['income', 'expense'] },
  icon: String,
  color: String
}
```

6) Alerts Collection

```
{
  user_id: { type: Schema.Types.ObjectId, ref: 'User' },
  budget_id: { type: Schema.Types.ObjectId, ref: 'Budget' },
  threshold_pct: { type: Number, default: 90 },
  triggered_at: { type: Date, default: null }
}
```

FRONT-END DEVELOPMENT:

Frontend Structure:

```
frontend/src/
├── components/
│   ├── Navbar.jsx           → Navigation bar with links and theme toggle
│   ├── TransactionForm.jsx  → Modal form for adding/editing transactions
│   ├── SummaryCard.jsx      → Reusable card for financial metrics
│   ├── SpendingPieChart.jsx → Recharts pie chart component
│   ├── SpendingLineChart.jsx → Recharts line chart component
│   ├── BudgetProgressBar.jsx → Visual budget progress bar
│   ├── ProtectedRoute.jsx   → Route guard for auth-only pages
│   └── ui/                  → Shared UI primitives (Button, Card, Input)
│
├── pages/
│   ├── Login.jsx            → User login page
│   ├── Register.jsx         → User registration page
│   ├── Dashboard.jsx        → Main dashboard with charts
│   ├── Transactions.jsx     → Transaction list with CRUD and CSV export
│   ├── Budgets.jsx          → Budget management with progress bars
│   ├── Accounts.jsx         → Financial account management
│   └── Settings.jsx         → User settings page
│
├── context/
│   ├── AuthContext.jsx      → JWT and login state management
│   └── ThemeContext.tsx     → Dark/Light theme state
│
├── services/
│   ├── api.js               → Axios instance with JWT interceptor
│   ├── transactionService.js → API calls for transactions
│   ├── budgetService.js     → API calls for budgets
│   └── dashboardService.js  → API calls for dashboard data
│
├── App.jsx                  → Root component with React Router
└── index.css                → Global styles and theme tokens
```

Pages:

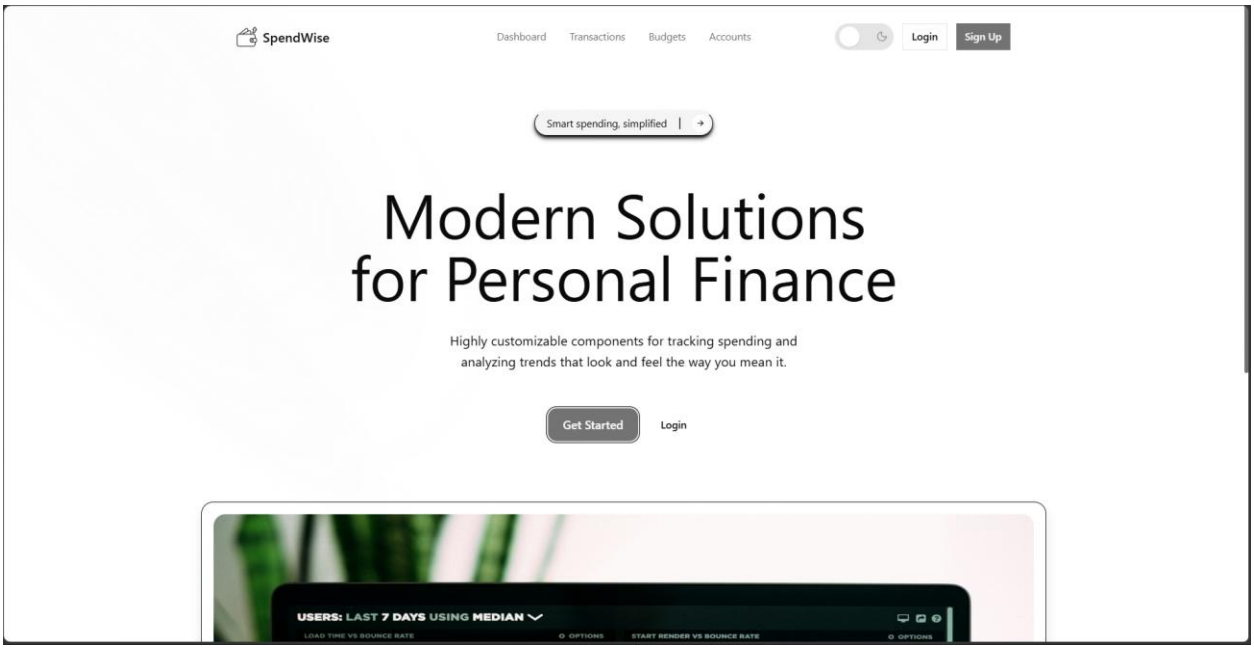
- **Login.jsx** → User login page with email and password form, JWT authentication, and secure redirect to the dashboard.
- **Register.jsx** → User registration page with name, email, and password fields, input validation, and automatic login on success.
- **Dashboard.jsx** → Main analytics dashboard displaying Total Income, Total Expenses, and Net Balance; a category-wise Pie Chart showing expense distribution; and a daily spending Line Chart tracking trends.
- **Transactions.jsx** → Full transaction management page with a data table and CRUD operations. Includes a "New Transaction" modal form and a "Export CSV" button.
- **Budgets.jsx** → Budget management page where users can set monthly spending limits for specific categories and view visual progress bars.
- **Accounts.jsx** → Financial account management page for creating and managing Bank, Cash, and Digital Wallet accounts.

Components (Reusable parts):

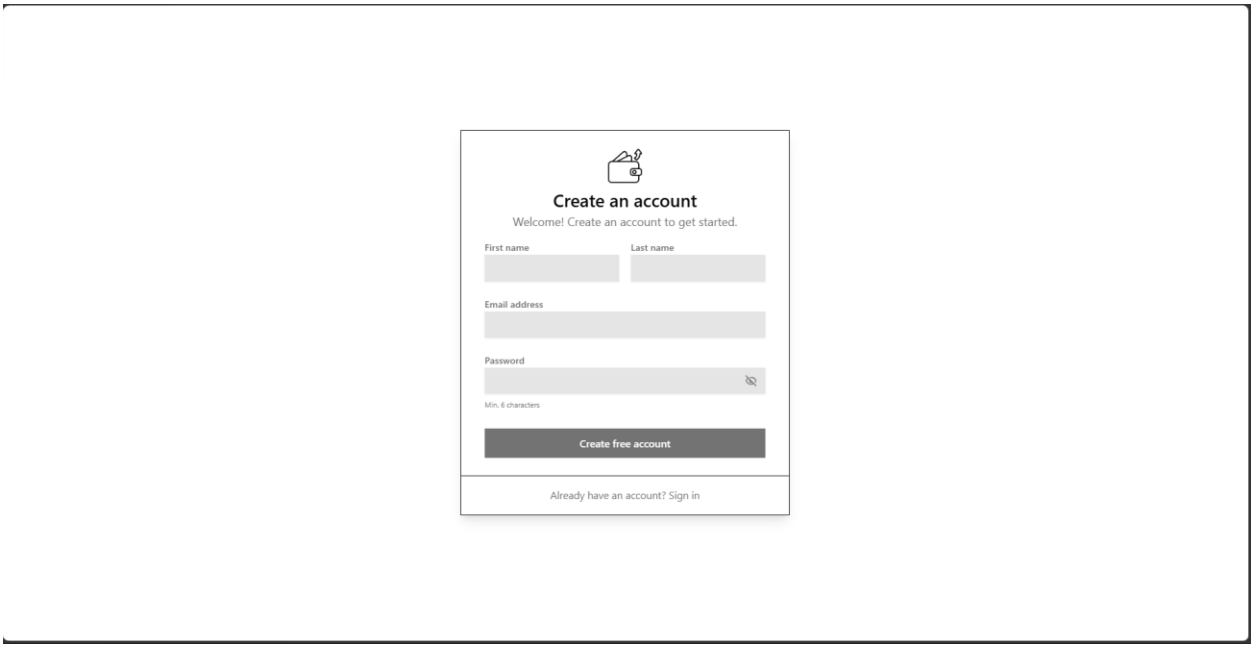
- **Navbar.jsx** → Top navigation bar with links, theme toggle switch, and user logout button.
- **TransactionForm.jsx** → Reusable modal form component for both creating and editing transactions.
- **SummaryCard.jsx** → Reusable card component for displaying formatted financial metrics with icons.
- **SpendingPieChart.jsx / SpendingLineChart.jsx** → Recharts-powered visualization components for spending analysis.
- **ui/** → Shared UI library containing Buttons, Cards, Inputs, and other base components.

Output Screenshots:

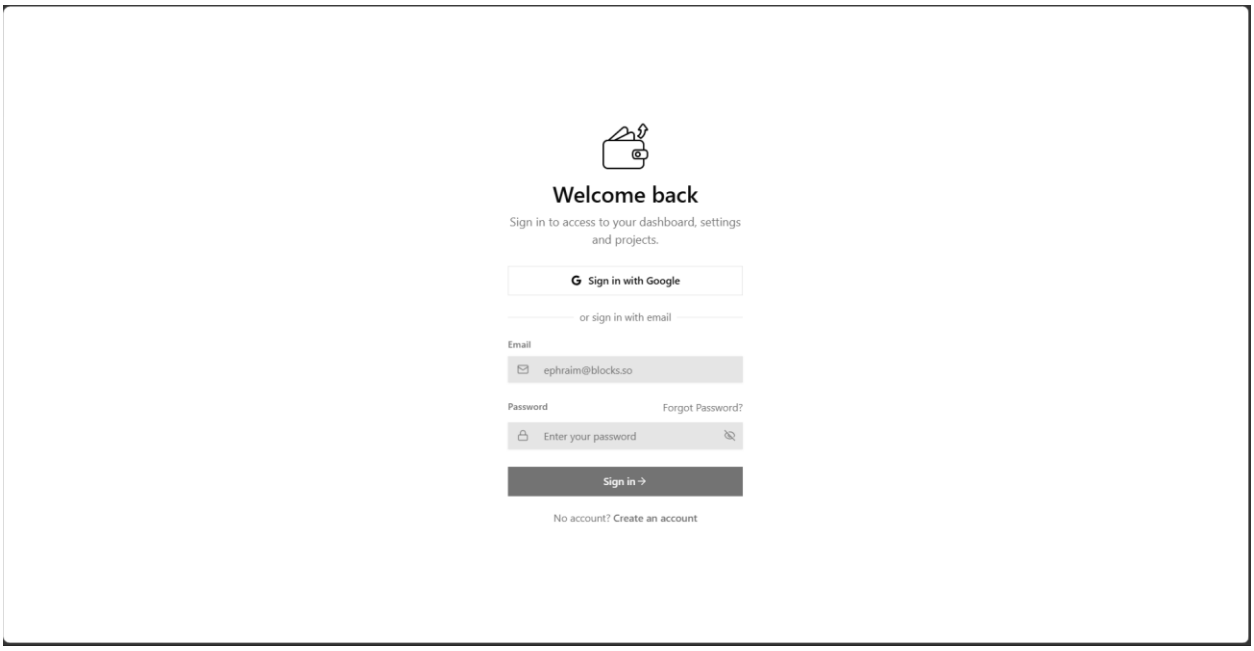
Landing Page:



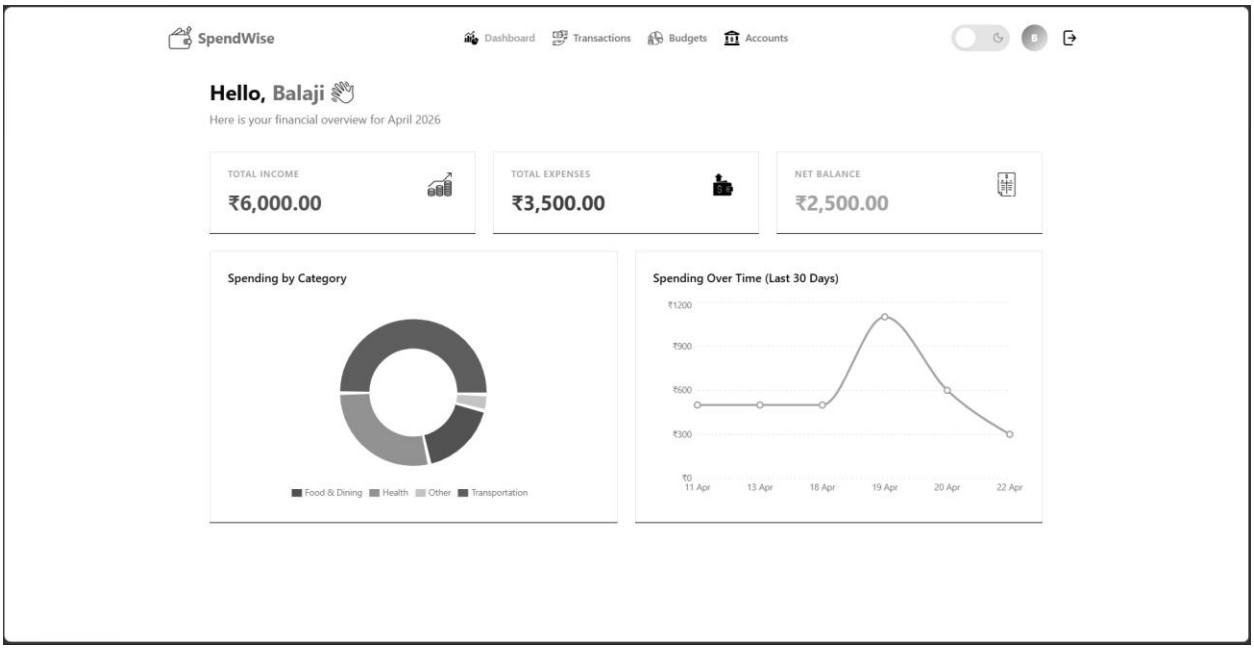
Register Page:



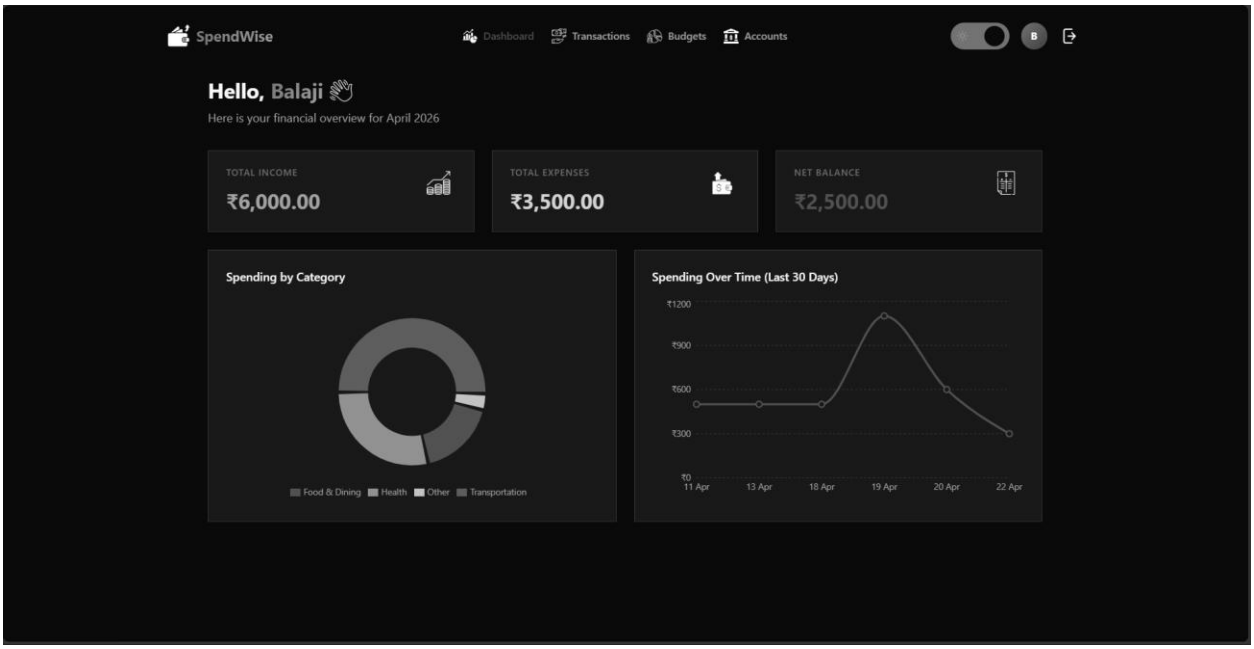
Login Page:



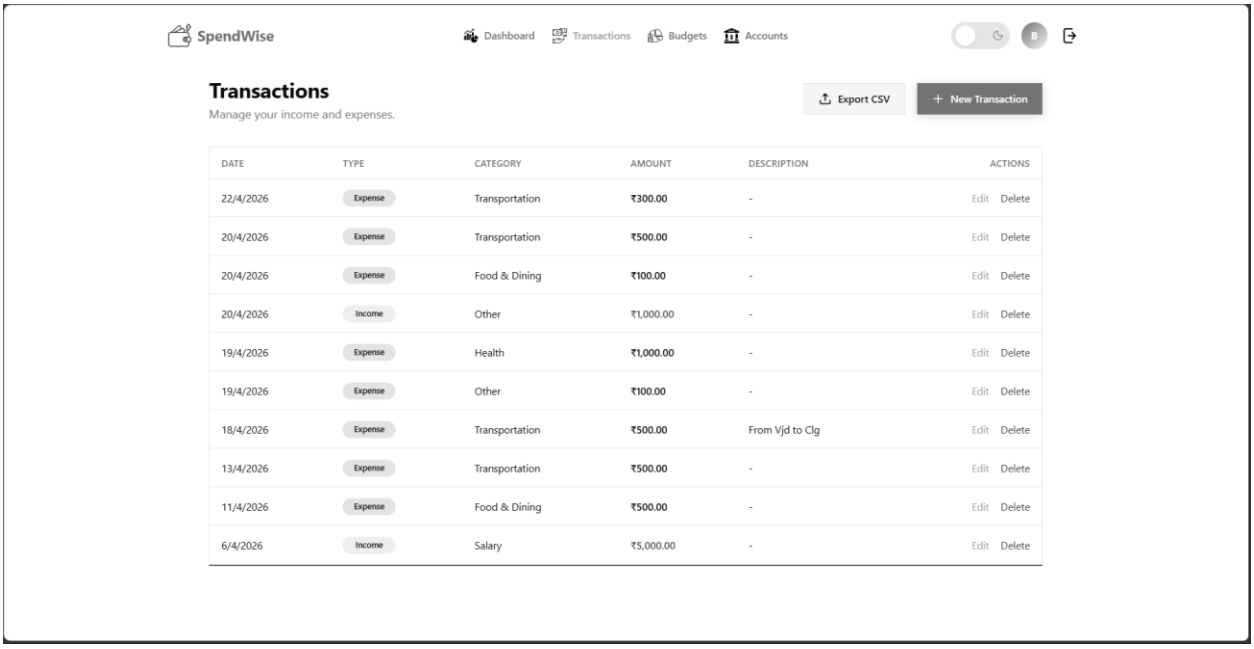
Dashboard (Light Mode):



Dashboard (Dark Mode):



Transactions Page:

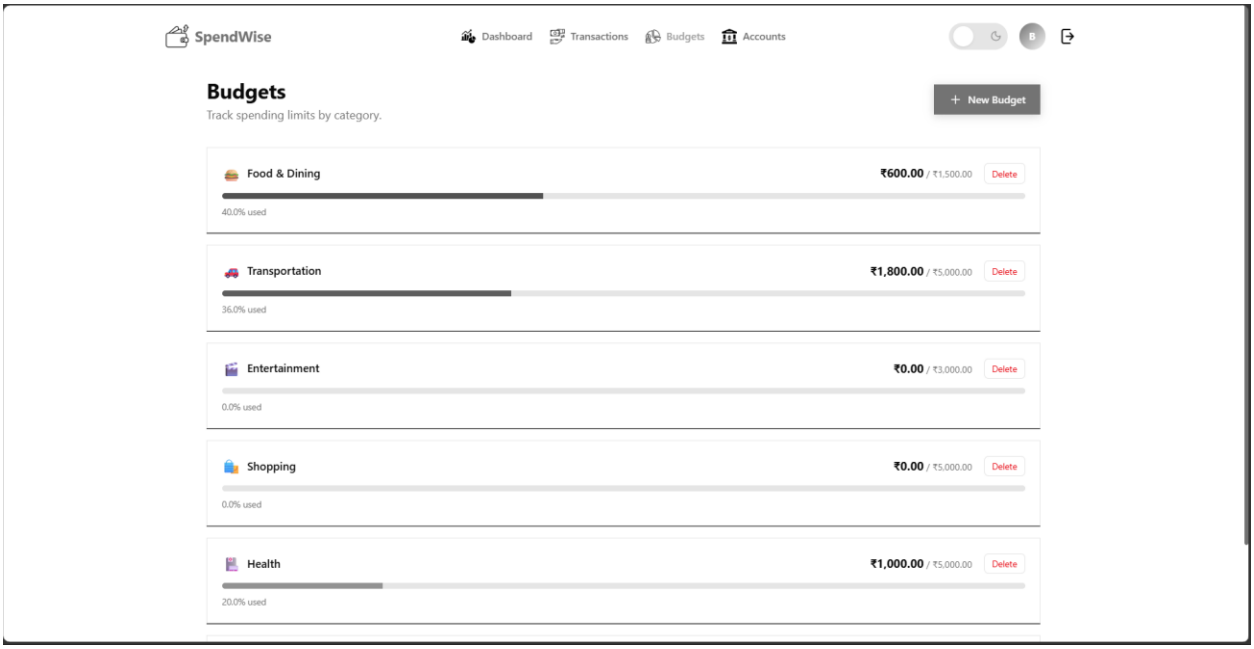


DATE	TYPE	CATEGORY	AMOUNT	DESCRIPTION	ACTIONS
22/4/2026	Expense	Transportation	₹300.00	-	Edit Delete
20/4/2026	Expense	Transportation	₹500.00	-	Edit Delete
20/4/2026	Expense	Food & Dining	₹100.00	-	Edit Delete
20/4/2026	Income	Other	₹1,000.00	-	Edit Delete
19/4/2026	Expense	Health	₹1,000.00	-	Edit Delete
19/4/2026	Expense	Other	₹100.00	-	Edit Delete
18/4/2026	Expense	Transportation	₹500.00	From Vjd to Clg	Edit Delete
13/4/2026	Expense	Transportation	₹500.00	-	Edit Delete
11/4/2026	Expense	Food & Dining	₹500.00	-	Edit Delete
6/4/2026	Income	Salary	₹5,000.00	-	Edit Delete

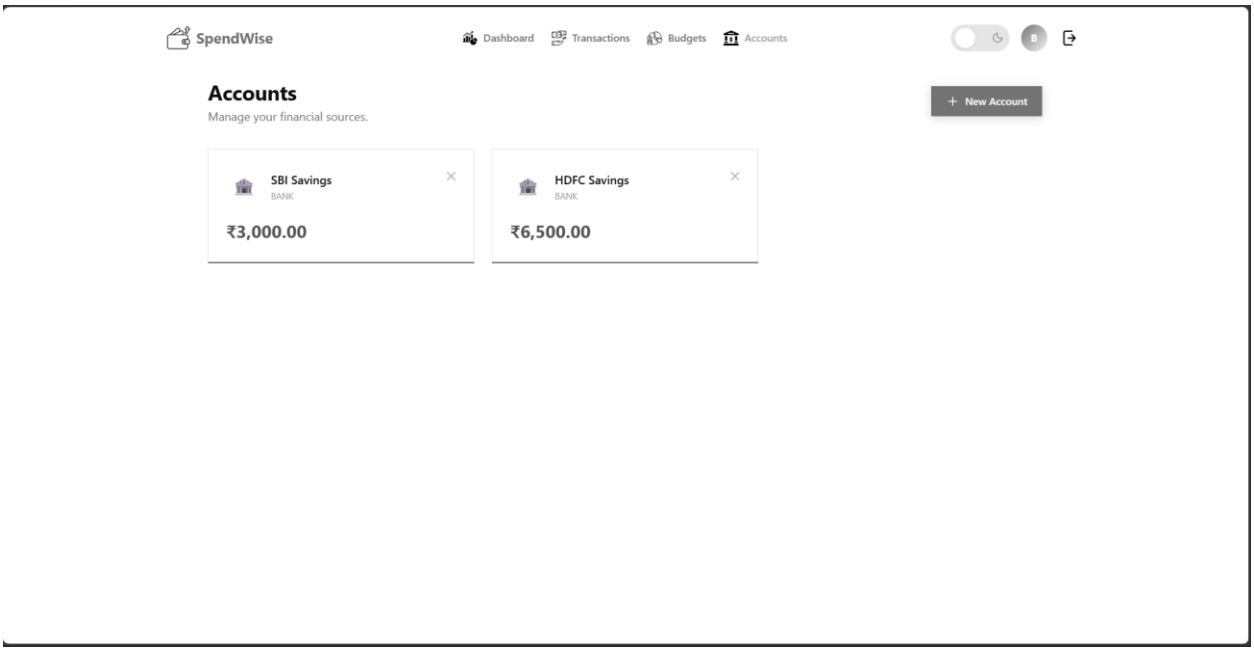
CSV Export:

	A	B	C	D	E	F
1	Date	Type	Category	Account	Amount	Description
2	22-04-2026	expense	Transportation	SBI Savings	300	
3	20-04-2026	expense	Transportation	SBI Savings	500	
4	20-04-2026	expense	Food & Dining	SBI Savings	100	
5	20-04-2026	income	Other	SBI Savings	1000	
6	19-04-2026	expense	Health	SBI Savings	1000	
7	19-04-2026	expense	Other	SBI Savings	100	
8	18-04-2026	expense	Transportation	HDFC Savings	500	From Vjd to Clg
9	13-04-2026	expense	Transportation	SBI Savings	500	
10	11-04-2026	expense	Food & Dining	SBI Savings	500	
11	06-04-2026	income	Salary	SBI Savings	5000	
12						

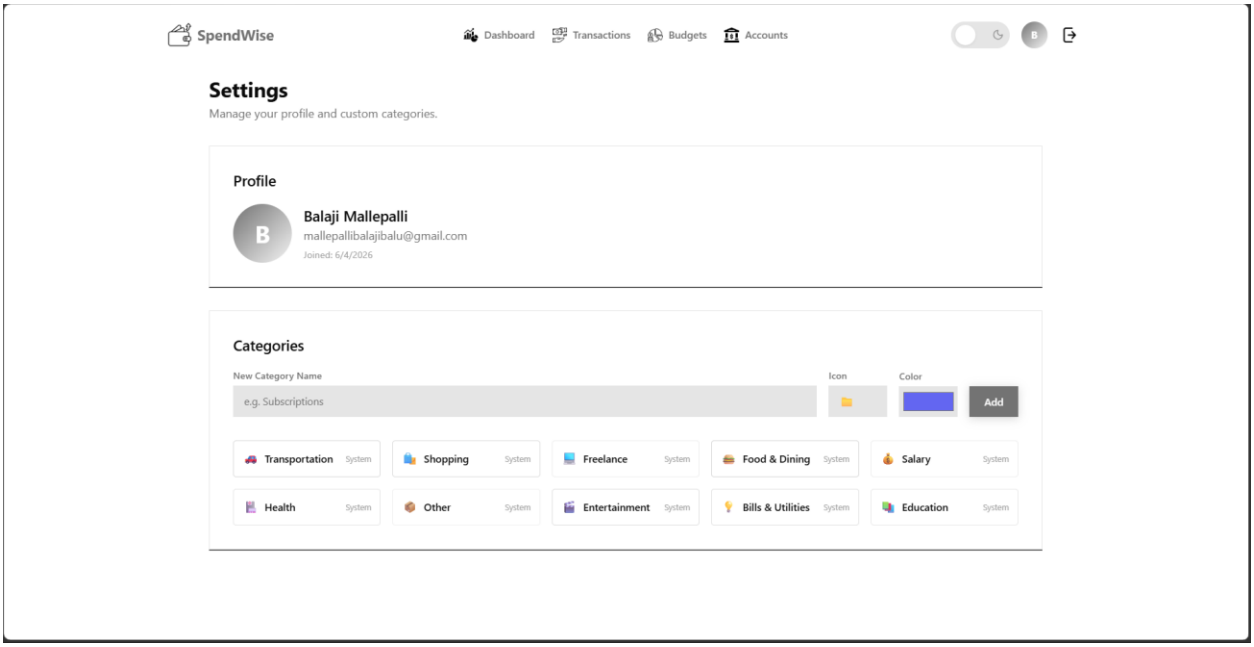
Budgets Page:



Accounts Page:



Profile/Settings Page:



Demo Video Link:

Drive Link :

https://drive.google.com/drive/folders/1mQGt6agQLdJh-3MSYyhY8EVI_WTa_AyW?usp=sharing

Code Repository Link:

GitHub:

https://github.com/balaji-mallepalli/SpendWise-Personal-Finance-Tracker-MERN

Live Deployment Links:

- Frontend (Vercel): https://spendwise-personal-finance-tracker-mern.vercel.app
- Backend (Render): https://spendwise-personal-finance-tracker-mern.onrender.com